

어제보다 더 진짜 같은 오늘을 렌더링하는 유택근입니다

Taekgeun You: Rendering a more realistic today than yesterday



더욱 현실감 넘치는 렌더링을 위해 끊임없이 고민하고 연구합니다.

학력

- 학사	컴퓨터공학과, 서강대학교	3.73 / 4.5	2018/03 ~ 2024/02
- 석사	컴퓨터공학과(그래픽스 연구실), 서강대학교	4.32 / 4.5	2024/03 ~ 2026/02

연구과제

- GPU 파이프라인 기반 TMIV 렌더러 고속화 및 확장기능 구현	2024/02 ~ 2024/11
- Adaptive Sampling for Ray tracing in res	2024/04 ~ 2025/04

기술

언어		API		Tools	
- C	★ ★ ★ ★ ☆	- OpenGL	★ ★ ★ ☆ ☆	- Git	★ ★ ★ ☆ ☆
- C#	★ ★ ☆ ☆ ☆	- Vulkan	★ ★ ★ ★ ☆	- Visual Studio	★ ★ ★ ★ ☆
- C++	★ ★ ★ ★ ☆	- OptiX	★ ★ ☆ ☆ ☆	- Unity	★ ★ ★ ☆ ☆
- Python	★ ★ ★ ☆ ☆	- CUDA	★ ★ ★ ☆ ☆	- Unreal	★ ☆ ☆ ☆ ☆
- glsl	★ ★ ★ ★ ☆				



유택근

YOU TAEK GEUN

1999.02.12

Email: xorrms9025@sogang.ac.kr

C.P: 010-8912-9025

어제보다 더 진짜 같은 오늘을 렌더링하는 유택근입니다

Taekgeun You: Rendering a more realistic today than yesterday



유택근

YOU TAEK GEUN

1999.02.12

Email: xorrms9025@sogang.ac.kr

C.P: 010-8912-9025

프로젝트

- Chat GPT와 Unity를 이용한 학습 게임 개발	2023/09 ~ 2023/12
- Unity를 이용한 VR 멀티 플레이어 게임 개발	2024/01 ~ 2024/03
- Vulkan과 CUDA를 활용한 선택적 레이 트레이서 개발	2024/09 ~ 2024/12
- CPU 레이 트레이서 구현	2024/10 ~ 2024/10
- OpenGL 기반 디퍼드 렌더러 구현	2024/12 ~ 2024/12
- OptiX 기반 레이 트레이서 구현	2024/12 ~ 2024/12
- Vulkan 기반 모바일 가우시안 레이 트레이서 개발	2025/05 ~ 2025/06

논문

- 모바일 플랫폼에서의 Vulkan 기반 광선 추적 렌더러의 성능 프로파일링	한국컴퓨터그래픽스 학회	2025/07
- 전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링	서강대학교(학위논문)	2026/02
- Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation (Best Poster Award 수상)	2026 ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games (I3D '26 Posters)	2026/05

특허

- METHODS OF IMPORTANCE METRIC-BASED, SELECTIVE FOVEADTED PROBE UPDATE FOR REPRESENTING DYNAMIC INDIRECT DIFFUSE LIGHT	진행 중
--	------

Research Adaptive Sampling for Ray tracing in res

Paper 1

모바일 플랫폼에서의 Vulkan 기반 광선 추적 렌더러의 성능 프로파일링

#Vulkan #C++ #RayTracing #RayQuery #DeferredRendering #HybridRendering
#ShadowMap #glTF #Mobile

내용

- 모바일 GPU 환경에서의 레이 트레이싱 성능 개선
- 디퍼드 렌더링 기반의 하이브리드 렌더러 개발
- Vulkan의 레이 트레이싱 파이프라인과 레이 쿼리 확장의 성능 비교
- 대용량 삼각형 및 다중 광원 환경에서의 레이 트레이싱 성능 최적화
- PBR 셰이딩 및 반사 / 굴절 / 그림자 효과 적용
- 쉐도우 맵과 레이 트레이싱 기반 그림자 효과 성능 비교

배운점

- Vulkan API에서의 메모리 할당 및 관리
- Vulkan API에서의 스왑체인 / 이미지 / 렌더 패스 구성
- 모바일 GPU를 위한 셰이더에서의 최적화
- GPU 및 셰이더 디버깅 및 프로파일링





Project 1 Chat GPT와 Unity를 이용한 학습 게임 개발

#Unity #C# #GPT #WebCrawling #팀프로젝트

목표

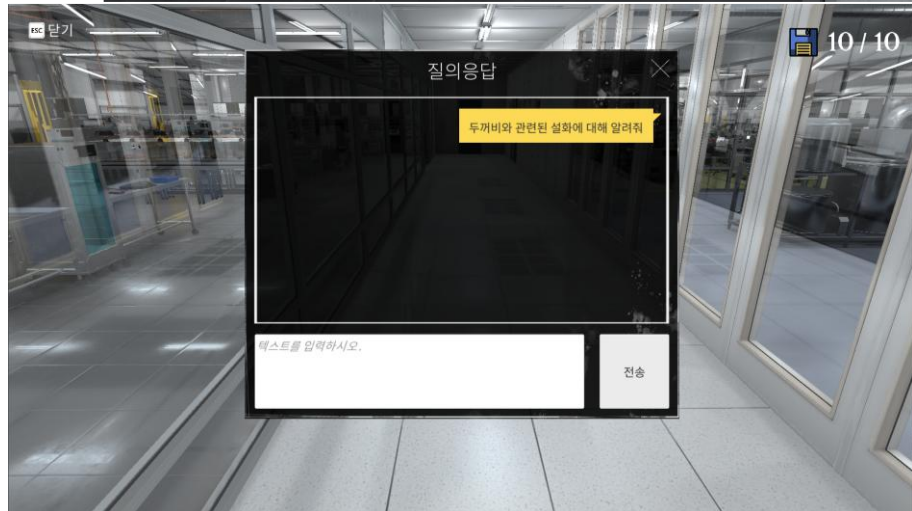
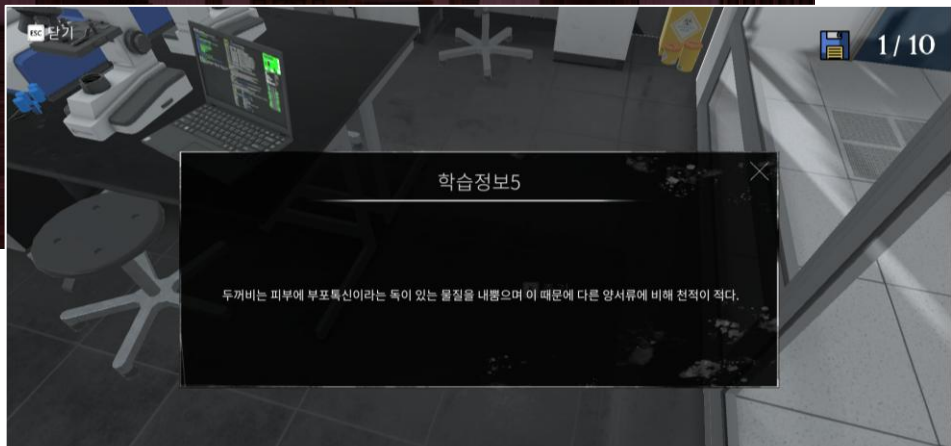
- Chat GPT 및 Unity 엔진을 활용한 방탈출 장르의 학습 게임 개발

내용

- 사용자가 입력한 학습 주제에 대한 위키피디아 웹 크롤링
- 웹 크롤링 데이터를 기반으로 Chat GPT를 통한 학습 정보 및 학습 문제/답안 생성
- 게임 내 오브젝트 습득을 통한 학습 정보 획득
- 모든 학습 정보 획득에 따른 퀴즈 응시 및 게임 클리어

역할

- UI 디자인 및 UI 로직 연결(Chat GPT와의 통신 포함)
- 위키피디아 웹 크롤링 구현
- 게임 내 메인 화면, 상점, 로딩 화면 구현
- 프로젝트 통합 및 디버깅
- 유저 설문조사 및 매뉴얼 제작
- 발표자료 제작, 보고서 작성, 회의 진행 및 회의록 작성



Project 2 Unity를 이용한 VR 멀티 플레이어 게임 개발

#Unity #C# #VR #팀프로젝트

목표

- Unity 엔진을 이용한 VR환경에서의 멀티 플레이어 게임 개발

내용

- VR 기기의 입력을 통한 게임의 동작 및 네트워크 활용 멀티 플레이 환경 구현
- VR 환경에서의 원활한 동작을 위한 LOD/메쉬/충돌 등의 최적화

역할

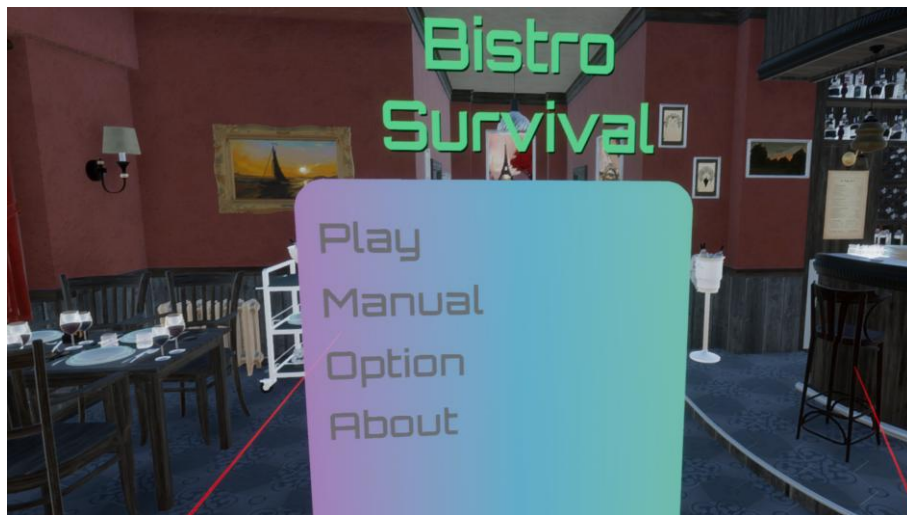
- UI 디자인 및 UI 로직 연결
- 모델 에셋 수정 및 멀티 플레이어 환경에서의 애니메이션 적용
- 플레이어 상황에 따른 동적 변화 배경음악 적용

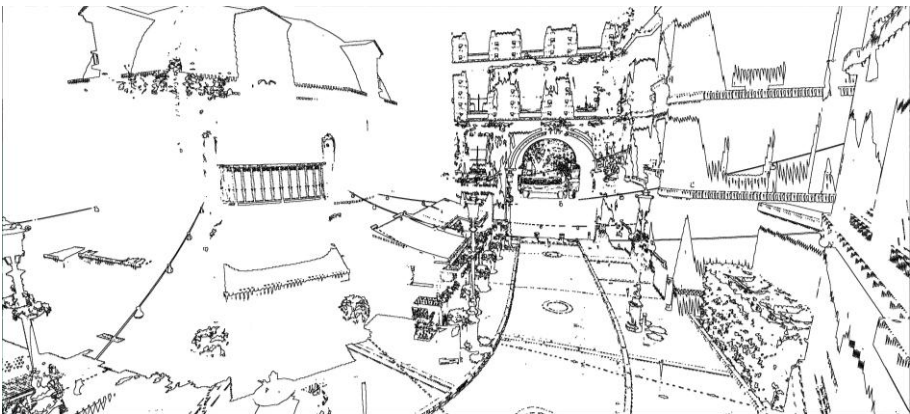
문제상황

- 시간적 여유로 인해 게임을 완성시키지 못함

배운점

- 게임 개발 과정에 있어 지속적인 개발 현황 파악 및 통합의 중요성
- 공동의 목표에 대한 구성원 간 협의의 중요성: 개인의 욕심을 줄이고, 공동의 목표를 위해 효율적인 방안 모색
- 지속적인 프로젝트 동작 확인 및 추가 기능은 동작 확인 후에 구현하는 것이 더 좋음





Project 3 Vulkan과 CUDA를 이용한 선택적 레이 트레이서 개발

#C++ #Vulkan #CUDA #RayTracer #ShadowMap #DeferredRendering #glTF #개인프로젝트

목표

- 그림자 효과 생성을 위한 레이 트레이싱 비용 감소 및 쉐도우 맵 그림자 품질 향상
→ 그림자 맵 렌더링 결과의 경계 부분에만 레이 트레이싱을 수행

내용

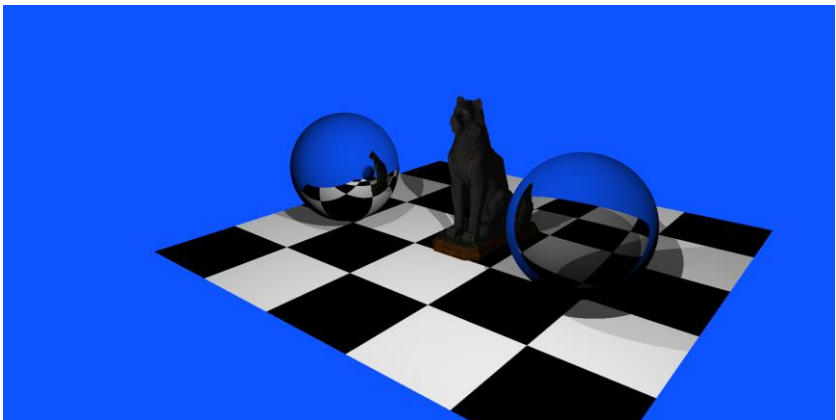
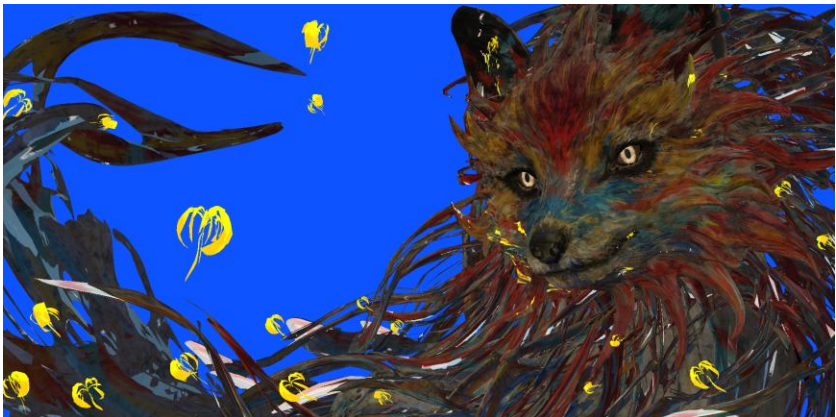
- Vulkan과 CUDA API 간의 interoperability를 활용:
그래픽 작업은 Vulkan, 병렬 처리 작업은 CUDA를 활용
- Vulkan API에서 생성한 쉐도우맵 텍스처에 대한 CUDA API를 통한 병렬 소벨 필터링
- 소벨 필터링 결과 엣지 이미지에 대한 선택적 레이 트레이싱 적용

문제상황

- 소벨 필터의 크기 문제로 의도한 결과처럼 깔끔한 그림자가 생성되지 않음

배운점

- Vulkan API에서 생성한 텍스처에 대한 CUDA API에서의 읽기/쓰기 방식 모드 설정의 중요성
- Vulkan API와 CUDA API에서의 메모리 공유 및 동기화(External Memory Use, Semaphore)



Project 4 CPU 레이 트레이서 개발

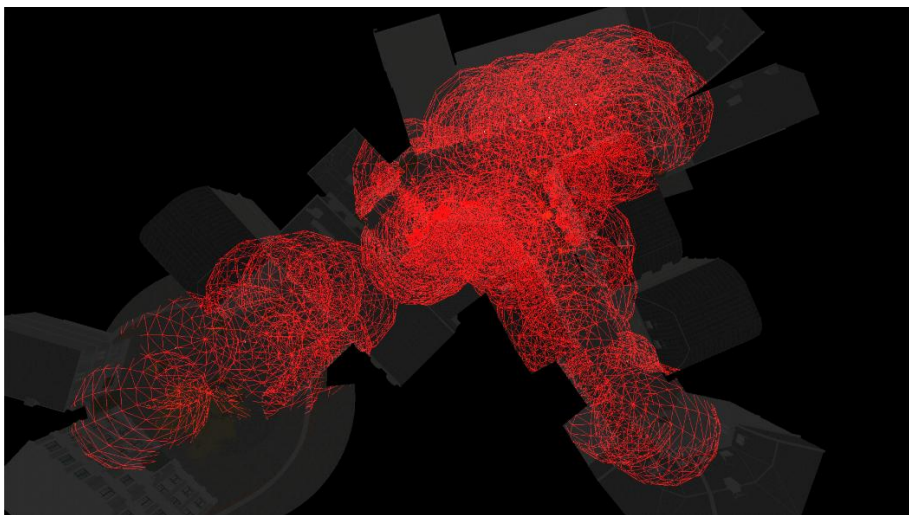
#C++ #RayTracing #glTF #개인프로젝트

목표

- CPU 환경에서 동작하는 간단한 레이 트레이서 개발

내용

- 삼각형, 사각형, 구체, AABB(Axis Aligned Bounding Box), 원뿔, 원기둥에 대한 충돌 검사 구현
- 레이 트레이싱을 통한 반사 / 굴절 / 그림자 효과 구현
- 광원 주변 무작위 지점에 대한 추가 광선 투사를 통한 부드러운 그림자(soft shadow) 효과 구현
- glTF 포맷 파일 로드를 통한 다양한 삼각형 메쉬 데이터 렌더링
- PBR 셰이딩 적용



Project 5 OpenGL 기반 디퍼드 렌더러 구현

#C++ #OpenGL #DeferredRendering #glTF #개인프로젝트

목표

- OpenGL API를 이용한 디퍼드 렌더러의 구현

내용

- OpenGL API를 이용한 디퍼드 렌더러의 단계적 구현

(1) 포워드 렌더러

(2) 디퍼드 렌더러

(3) 디퍼드 렌더러 + 셰이더 내 반복문의 분기를 통한 선택적 셰이딩

(4) 디퍼드 렌더러 + 광원 영향 범위 크기의 구체 렌더링 및 스텐실 버퍼를 통한 최적화

- PBR 셰이딩 적용

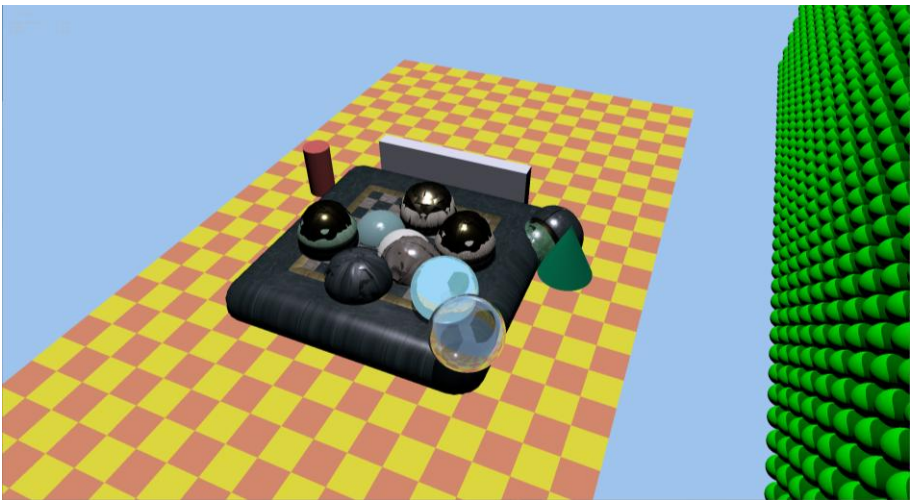
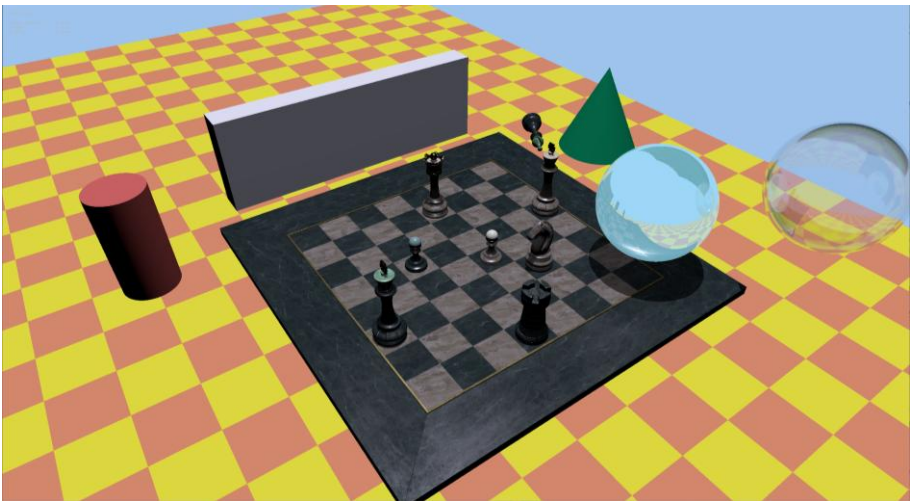
- glTF 포맷 파일 로드를 통한 다양한 삼각형 메쉬 데이터 렌더링

구현 상세

- 빛의 감쇄 효과를 통한 광원 영향 범위 설정

- 구체 렌더링 및 깊이 값 비교를 통한 스텐실 버퍼 write

- 스텐실 값 비교 과정을 통한 선택적 픽셀 셰이딩 및 결과 이미지 블렌딩 적용



Project 6 OptiX 기반 레이 트레이서 구현

#C++ #OptiX #RayTracing #glTF #개인프로젝트

목표

- OptiX API를 활용한 레이 트레이서 구현 및 동적 물체 지원

내용

- Optix API 기본 프리미티브 타입: 삼각형, 구체, AABB(Axis Aligned Bounding Box)에 대한 레이 트레이싱
- 커스텀 물체(원뿔, 원기둥)에 대한 충돌 구현
- 레이 트레이싱을 통한 반사 / 굴절 / 그림자 효과 적용
- PBR 셰이딩 적용
- 하나의 물체에 대한 인스턴싱 및 다중 렌더링 적용
- glTF 포맷 파일 로드를 통한 다양한 삼각형 메쉬 데이터 렌더링
- 동적 물체에 대한 업데이트 및 레이 트레이싱 적용

Project 7 Vulkan 기반 모바일 가우시안 레이 트레이서 개발

#C++ #Vulkan #RayTracing #RayQuery #GaussianSplatting #Mobile #팀프로젝트

목표

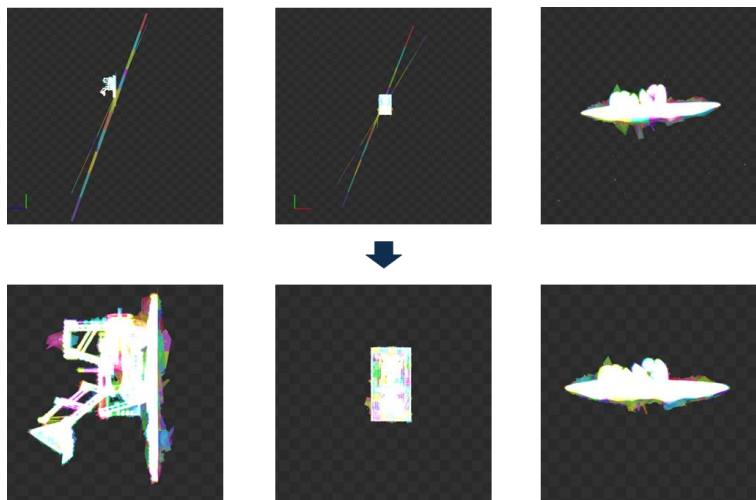
- 모바일 환경에서 동작하는 Vulkan 기반 가우시안 레이 트레이서 개발 및 최적화

내용

- NVIDIA의 3DGRUT 코드를 Vulkan 기반 환경으로 포팅
- Vulkan의 레이 트레이싱 파이프라인과 레이 쿼리 확장을 통해 모두 구현 및 성능 비교
- 훈련 과정에서 생기는 잘못된 가우시안 파티클 제거

구현 상세

- 하나의 BVH(공간 가속 구조)를 세분화하여 3D cell 형태로 분할함
- 레이 쿼리 활용 시의 최적화:
 - (1) Shared memory에 가우시안 파티클 정렬 결과 저장
 - (2) 셰이더 내에 있던 loop를 host의 command buffer 기록 부분으로 옮김
(이에 따라 각 쓰레드에서는 전체 BVH에 대해 한 번의 순회 및 정렬을 수행하게 됨)
 - (3) Transmittance가 남은 픽셀만 모아 레이 쿼리(레이 트레이싱) 수행



Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

#Vulkan #C++ #RayTracing #RayQuery #DeferredRendering #HybridRendering #DDGI #glTF #Mobile



목표

- Vulkan API를 활용한 모바일 환경에서 동작하는 레이 트레이싱 및 DDGI 기반 전역 조명 렌더러 구현 및 최적화

내용

- Whitted-style 레이 트레이서 최적화:

- (1) 레이 트레이싱 파이프라인 셰이더 컴파일 방식 최적화
- (2) 레이 쿼리 확장 활용
- (3) 저용량 텍스처 활용
- (4) 하이브리드 레이 트레이서의 Geometry Pass 최적화

- DDGI 카메라 중심 다중 볼륨 구성 및 적응적 갱신 방식 적용:

- (1) 카메라 이동에 따라 새로 생성되는 프로브
- (2) 시각적으로 중요한 프로브
- (3) 라운드 로빈에 따른 순차 갱신 프로브

Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

렌더러 구조

```
void PIKAFullRayqueryRenderer::createForwardPassComputePipeline()  
{  
    forwardPassCompute = std::make_unique<ComputePass>(  
        vulkanDevice,  
        renderContext,  
        resourceManager,  
        assetLoader);  
  
    ComputePass::PipelineContext forwardPassComputePipelineContext;  
  
    forwardPassComputePipelineContext.shaderEntry = ... ;  
  
    forwardPassCompute->createPipeline(  
        forwardPassComputePipelineContext,  
        shaderModules);  
}
```

파이프라인 생성 코드 예시

내용

- 렌더링 방식에 따른 렌더러 구분 및 렌더 매니저를 통한 관리
- 사용 파이프라인에 따른 Graphics Pass / Ray Tracing Pass / Compute Pass 모듈화
각 pass마다 파이프라인이 바인딩되어 있음
- 각 렌더러가 렌더링 방식에 따라 다수의 pass 객체 및 descriptor set을 관리함

부족한 점

- 싱글톤 패턴을 적용하지 않고, 각 클래스마다 필요한 생성자를 주입하는 방식을 활용하여 코드가 난잡해짐
- 게임엔진 구성에 대해 이해가 부족하여 Engine/Core 등으로 구조를 나누지 않음
- 렌더러 기능이 많아지며 각 기능별 Class의 기능이 혼재됨, 특정 Class에 부담이 커짐

Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

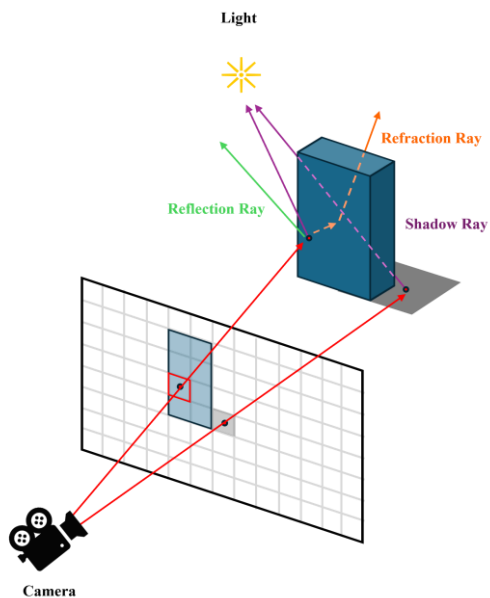
Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

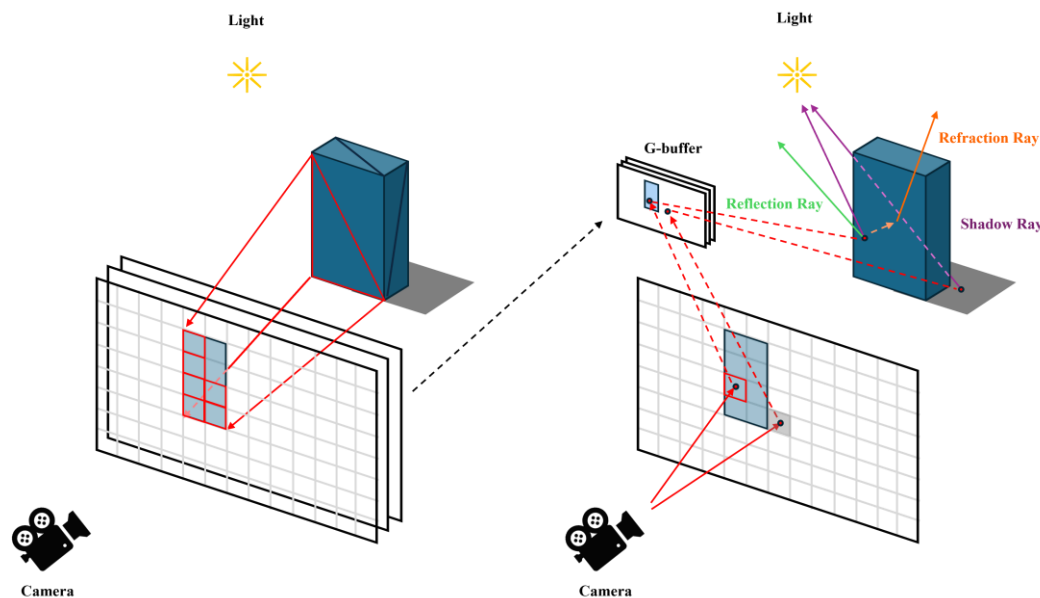
Whitted-style 레이 트레이서 구현

내용

- Vulkan의 레이 트레이싱 파이프라인 및 레이 쿼리 확장을 이용한 레이 트레이서 구현
- 레이 트레이싱의 주 광선 투사를 래스터화로 대체한 디퍼드 렌더링 기반 하이브리드 레이 트레이서 구현



풀 레이 트레이싱



하이브리드 레이 트레이싱

Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

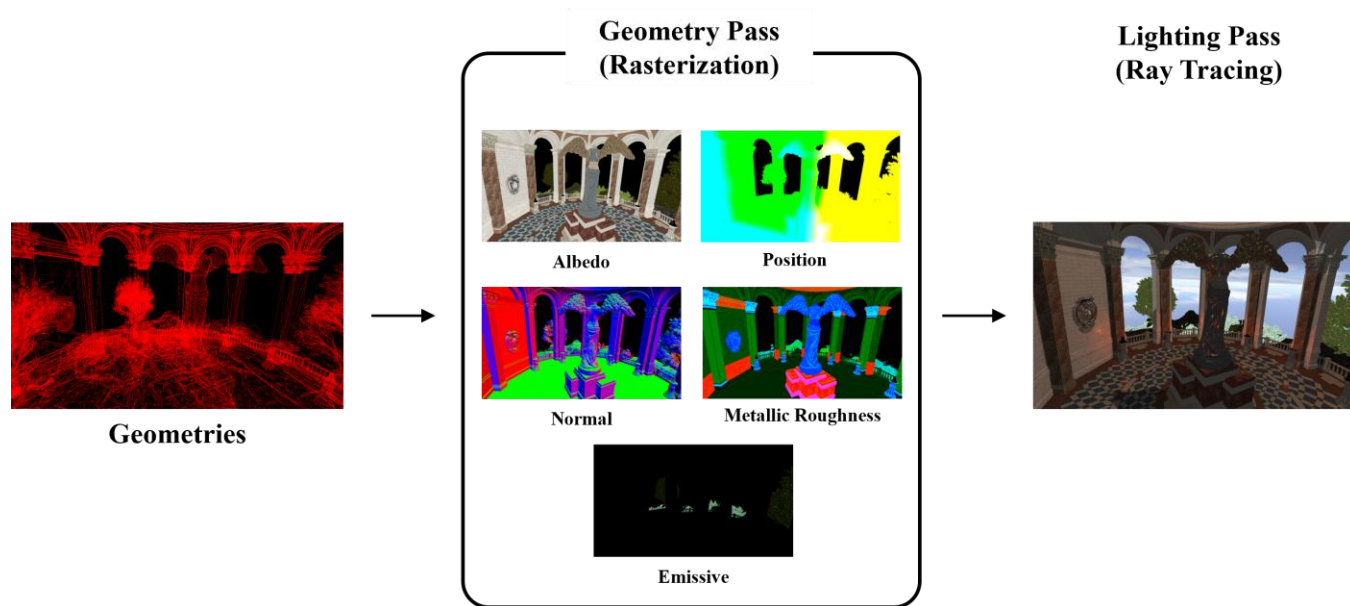
Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

Whitted-style 레이 트레이서 구현 (계속)

내용

- Vulkan의 레이 트레이싱 파이프라인 및 레이 쿼리 확장을 이용한 레이 트레이서 구현
- 레이 트레이싱의 주 광선 투사를 래스터화로 대체한 디퍼드 렌더링 기반 하이브리드 레이 트레이서 구현



하이브리드 레이 트레이서 파이프라인

Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

Whitted-style 레이 트레이서 최적화

레이 트레이싱 파이프라인 확장 컴파일 방식 최적화

- RayGen 셰이더에서만 레이 트레이싱을 수행할 경우 각 셰이더가 통합되어 컴파일됨
- 셰이더 통합 컴파일에 따라 각 모듈의 호출 오버헤드 감소
- 풀 레이 트레이서에서 최대 2.74x, 하이브리드 레이 트레이서에서 최대 1.75x 성능 향상

레이 쿼리 확장의 활용

- 간단한 레이 트레이싱 수행 시 레이 쿼리 확장이 더 좋은 성능을 보임
- 그래픽스 파이프라인 및 컴퓨트 파이프라인 활용의 경우 상황에 따라 달라 프로파일링 필요

저용량 압축 텍스처의 활용

- KTX2 포맷을 활용해 ETC1S 모드의 5레벨 텍스처 압축 수행: 약 1/10 크기로 압축
- 캐시 메모리가 부족한 모바일 환경에 효과적
- 하이브리드 레이 트레이서의 래스터화 단계에서 특히 좋은 효율을 보임
- 풀 레이 트레이서에서 최대 1.07x, 하이브리드 레이 트레이서에서 최대 1.08x 성능 향상

Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

하이브리드 레이 트레이서의 Geometry Pass 최적화

Bindless Binding 구조

- 기존: Geometry 단위로 Descriptor Set을 할당하여 반복적으로 Draw Call 호출하는 구조
- 변경: 하나의 Descriptor Set에 모든 리소스 정보를 담고, 셰이더 내에서 간접 인덱싱을 수행하는 Bindless Binding 구조로 변경
- 결과: 최대 약 50%의 성능 감소
- 원인: 간접 인덱싱 과정에서의 과도한 메모리 접근 오버헤드로 추측

Texture Sampling 방식 변경

- 기존: 0 Level Mip에 대해 Bilinear Filtering 활용
- 변경: Trilinear Filtering 활용
- 결과: 최대 약 1.78x 성능 향상
- 원인: 적절한 mip 레벨 선택에 따라 작은 해상도의 텍스처에 접근하게 되며 캐싱 효율 및 메모리 접근 비용 감소

Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

하이브리드 레이 트레이서의 Geometry Pass 최적화 (계속)

Indirect Draw Call 활용 Two-pass Occlusion Culling

- 알고리즘:

- (1) 이전 프레임에 렌더링된 물체를 마킹 후 View Frustum Culling을 수행해 Indirect Draw 명령을 구성
- (2) Indirect Draw Call 호출을 통해 Depth Map을 생성
- (3) Depth Map을 통해 HZB(Hierarchical Z-Buffer)를 구성
- (4) HZB 및 View Frustum Culling을 통해 Indirect Draw 명령 구성 및 Indirect Draw Call 호출을 통한 최종 렌더링



HZB 생성 과정

Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

하이브리드 레이 트레이서의 Geometry Pass 최적화 (계속)

Indirect Draw Call 활용 Two-pass Occlusion Culling (계속)

- 초기엔 Indirect Draw Count 함수를 활용했으나, 확장 함수임에 따라 제대로 최적화되지 않은 성능 양상을 보임
이에 따라 Indirect Draw 함수를 사용하되, 명령을 구성할 때 차폐되는 물체에 대해서는 instance count를 0으로 설정하여 렌더링하지 않도록 함
- 결과: 최대 약 12% 가량 성능이 감소함
- 원인: 사용한 에셋의 Geometry 수가 많지 않아 Occlusion Culling 오버헤드가 더 큰 것으로 추정

Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

하이브리드 레이 트레이서의 Geometry Pass 최적화 (계속)

```
void vkglTF::Model::sortAndCullPrimitives(glm::vec4 frustumPlanes[6], glm::vec3 camPos)
{
    sortedPrimitives.clear();

    std::vector<vkglTF::Primitive*> primitives;
    if (isDynamicModel)
        primitives = linearNodes[currentModelIndex]->mesh->primitives;
    else
        primitives = linearPrimitives;

    for (const auto primitive : primitives)
    {
        vkglTF::Primitive _primitive = *primitive;
        _primitive.dimensions.center = glm::vec3(modelTransformUniformData.modelMatrix *
                                                  glm::vec4(primitive->dimensions.center, 1.0f));
        if (sphereInFrustum(_primitive.dimensions.center, _primitive.dimensions.radius, frustumPlanes))
        {
            sortedPrimitives.push_back(_primitive);
        }
    }

    std::sort(sortedPrimitives.begin(), sortedPrimitives.end(),
              [&](const Primitive& a, const Primitive& b) {
                  float da = distanceSquared(a.dimensions.center, camPos);
                  float db = distanceSquared(b.dimensions.center, camPos);
                  return da < db; // front-to-back
              });
}
```

View Frustum Culling 및 정렬 함수

Early-Z Test 및 CPU 드로우 순서 정렬

- 셰이더 상에서 Early-Z Test를 Enable
- CPU 상에서 물체의 중심과 최대 반지름 길이를 활용해 구체에 대한 View Frustum Culling 수행 및 물체의 중심을 기준으로 정렬하여 정렬 순서대로 Draw Call 호출
- 결과: 최대 약 1.36x 성능 향상

최종 성능 비교

- 풀 레이 트레이서: 최대 144.66% 성능 향상
- 하이브리드 레이 트레이서: 최대 93.86% 성능 향상

Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

DDGI 카메라 중심 다중 볼륨 구성 및 적응적 갱신 방식 적용

DDGI

- 장면에 3차원 격자 형태로 프로브(probe)를 배치함
- 프로브는 조도를 수집하기 위한 개념으로, 매 프레임마다 구면 방향으로 레이 트레이싱을 수행해 Radiance를 저장하고, 고정된 방향으로 코사인 가중치를 취해 Irradiance를 저장
- 셰이딩 시에는 주변 프로브 8개에 대해 표면의 Normal 방향으로 Irradiance 값을 샘플링하고 보간하여 Indirect Diffuse Global Illumination 계산

기존 DDGI의 문제

- 장면 규모가 커질수록 배치해야 하는 프로브가 증가함에 따라 레이 트레이싱 비용이 크게 증가
- 비활성 프로브에서도 매 프레임마다 일정 비율의 레이 트레이싱 수행
- 레이 트레이싱 성능이 제한적인 모바일 환경에는 특히 부담됨

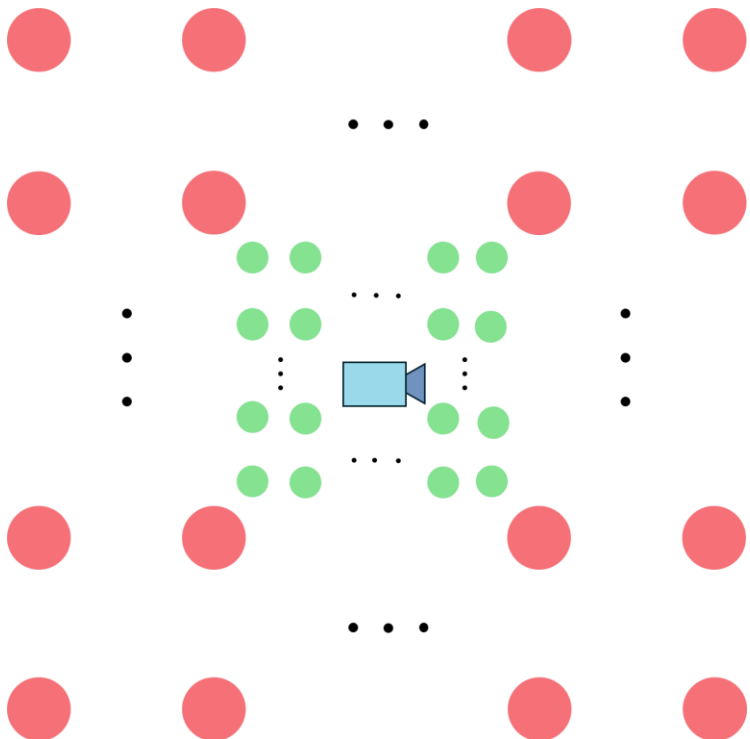
Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

DDGI 카메라 중심 다중 볼륨 구성 및 적응적 갱신 방식 적용(계속)



카메라 중심 다중 볼륨 구성

제안 방식

- 카메라를 기준으로 다중 볼륨 구성 및 전체 프로브 갱신 예산 부여:
프로브 볼륨 당 200개의 프로브 갱신 예산 부여
- 중요도 및 라운드 로빈 기반 적응적 프로브 갱신
 - (1) 카메라 이동에 따라 생성된 프로브 무조건 갱신
 - (2) 남은 예산의 2/3를 중요 프로브에 할당
 - (3) 남은 예산은 Round-robin 순차 갱신 프로브에 할당

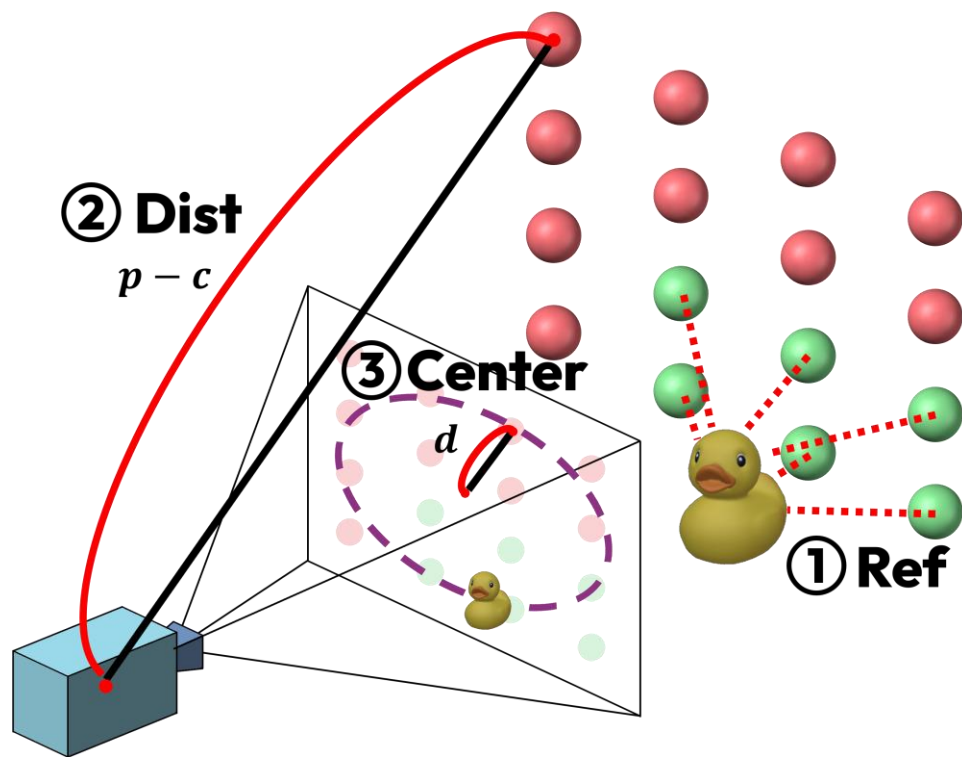
Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

DDGI 카메라 중심 다중 볼륨 구성 및 적응적 갱신 방식 적용(계속)



중요도 지표의 3요소

프로브 중요도 지표

$$\rho = w_{ref} \cdot w_{dist} \cdot w_{center}$$

① w_{ref} : probe usage indicator

$$w_{ref} = \begin{cases} 1.0, & \text{if referenced,} \\ w_{ref}^{min}, & \text{otherwise.} \end{cases}$$

② w_{dist} : probe's proximity to camera [1]

$$w_{dist} = \begin{cases} 1.0, & \text{if } \|p - c\| < k. \\ e^{-(\|p - c\| - k)}, & \text{otherwise.} \end{cases}$$

③ w_{center} : probe's proximity to user's focal center

$$d = \left(\frac{X_{clip}}{W_{clip}}, \frac{Y_{clip}}{W_{clip}} \right), \quad n_d = \text{clamp}(\|d\|, 0.0, 1.0)$$

$$w_{center} = \max(1.0 - \text{smoothstep}(0.7, 1.0, n_d), w_{center}^{min})$$

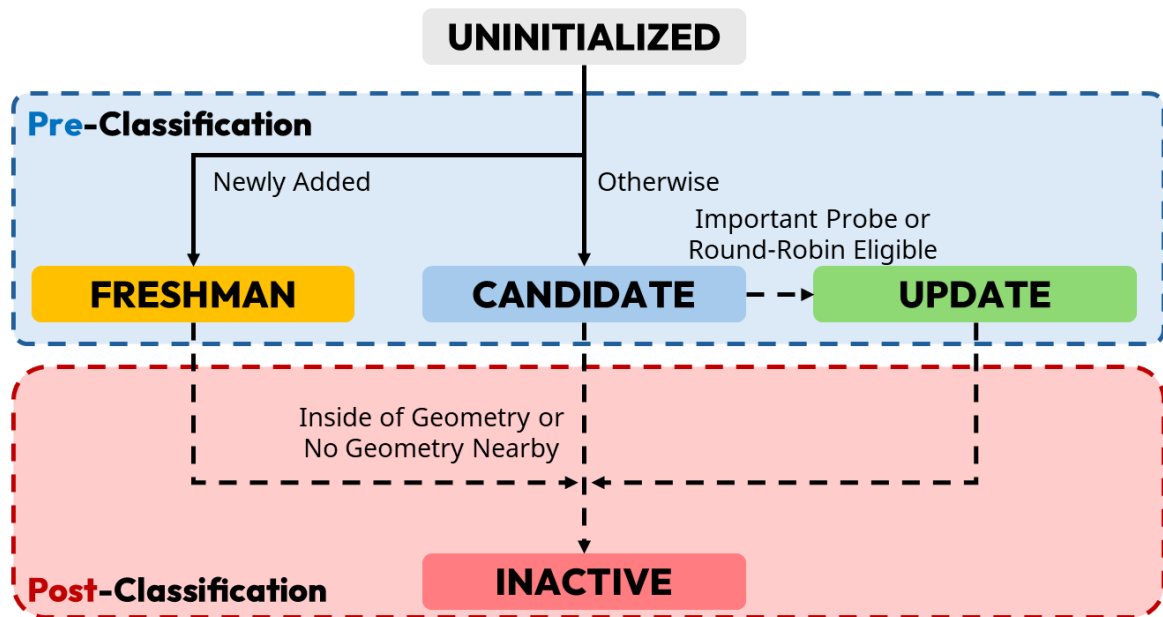
Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

DDGI 카메라 중심 다중 볼륨 구성 및 적응적 갱신 방식 적용(계속)



프로브 상태 머신

프로브 상태 머신

- 신규(FRESHMAN): 카메라 이동 방향으로 새로 생성된 프로브
- 후보(CANDIDATE): 신규 프로브 제외 나머지 프로브로 사전 분류 단계에서 우선도 및 라운드 로빈 순서에 따라 갱신 프로브로 전이되거나 사후 분류 단계에서 비활성 프로브로 전이
- 갱신(UPDATE): 이번 프레임에 갱신을 수행하는 프로브
- 비활성(INACTIVE): 갱신 및 웨이딩에서 배제되는 프로브

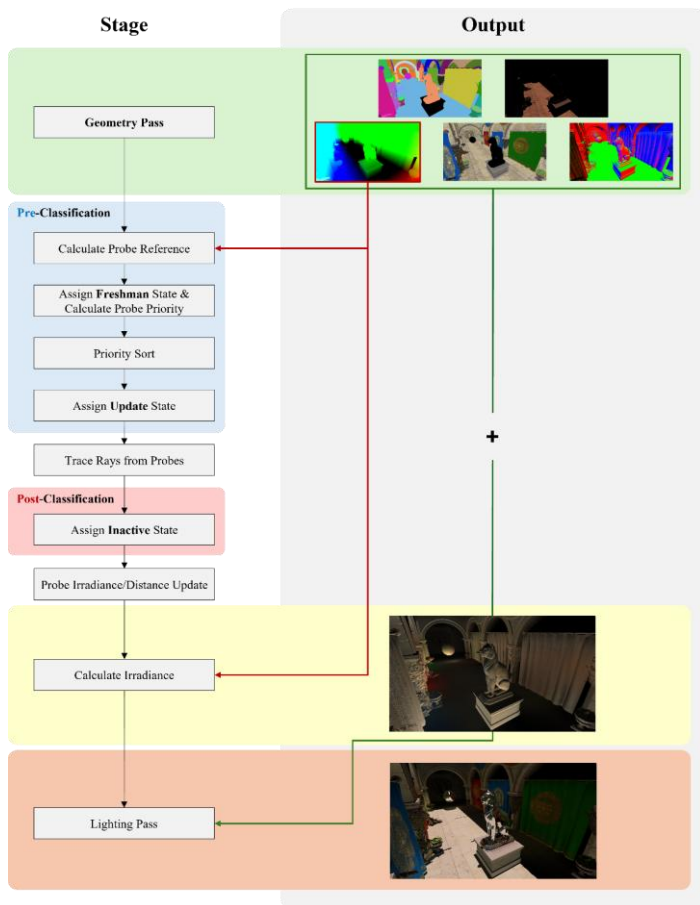
Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

DDGI 카메라 중심 다중 볼륨 구성 및 적응적 갱신 방식 적용(계속)



적응적 갱신 파이프라인

적응적 갱신 파이프라인

- 전체 파이프라인:

- (1) Geometry Pass
- (2) 프로브 참조 여부 확인(쉐이딩에 관여하는지)
- (3) 우선도 계산 및 정렬
- (4) 프로브 레이 트레이싱
- (5) 프로브 분류
- (6) 프로브 갱신
- (7) 간접 조명 계산
- (8) 최종 쉐이딩

- 프로브 참조 여부를 계산하기 위해 디퍼드 렌더링을 수행

- 디퍼드 렌더링을 통해 획득한 Position 텍스처를 통해 참조되는 프로브에 마킹을 수행

- 전체 해상도로 계산하는 것은 계산 오버헤드가 매우 크므로, 1/16 해상도로 수행

- 조도를 계산하는 부분도 오버헤드를 줄이기 위해 1/4 해상도로 수행

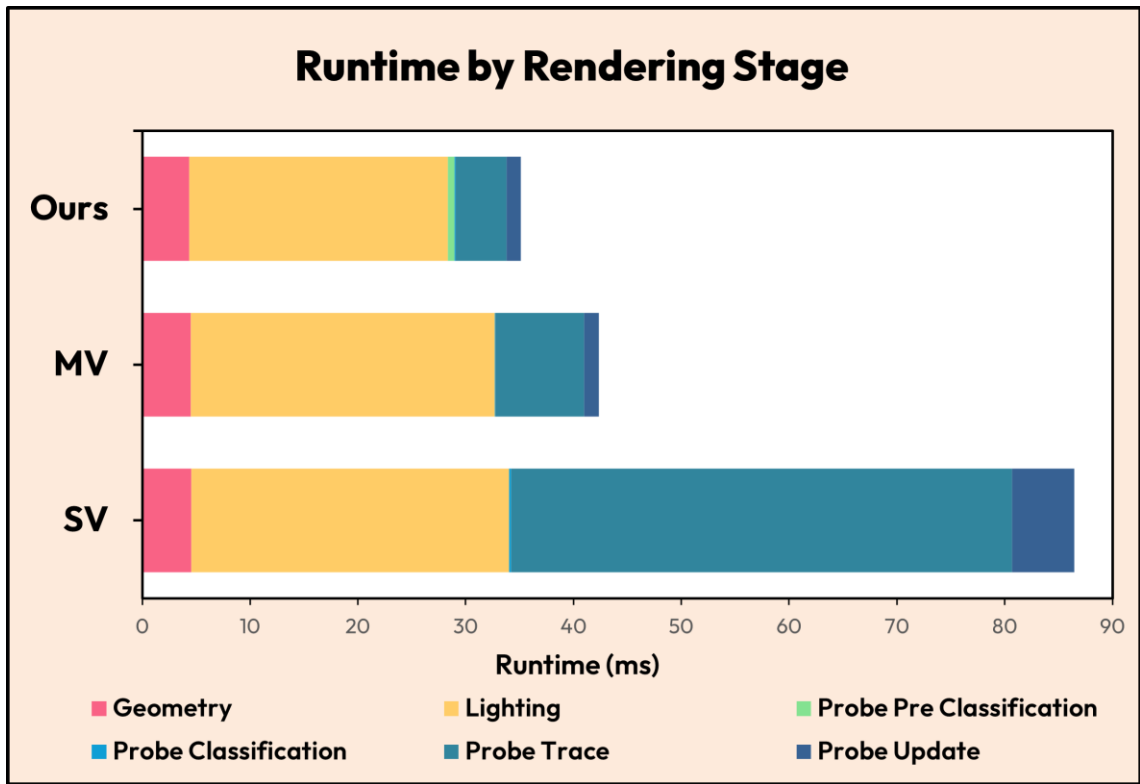
Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

DDGI 카메라 중심 다중 볼륨 구성 및 적응적 갱신 방식 적용(계속)



최종 성능 비교

- SV(Static Volume): 장면에 고정적인 볼륨을 배치 및 이진 상태 분류 체계
- MV(Moving Volume): 카메라 부착 다중 볼륨 및 이진 상태 분류 체계
- 풀 레이 트레이서
BISTRO: 26.8 fps / SPONZA: 51.5 fps (SV 대비 각각 2.5x, 1.62x)
- 하이브리드 레이 트레이서
BISTRO: 30.7 fps / SPONZA: 64.4 fps (SV 대비 각각 2.63x, 1.7x)

BISTRO 장면에 대한 하이브리드 레이 트레이서 성능 비교

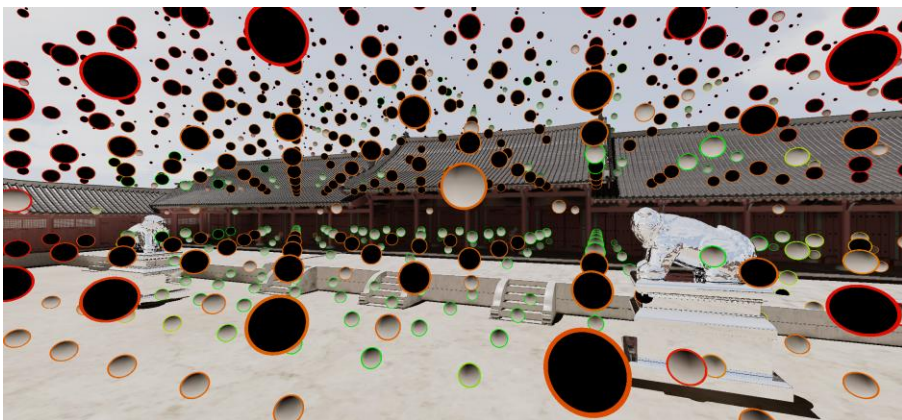
Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

DDGI 카메라 중심 다중 볼륨 구성 및 적응적 갱신 방식 적용(계속)



프로브 중요도 시각화: 0.0 (빨강) ~ 1.0 (초록)



적용 결과 (SUNTEMPLE)

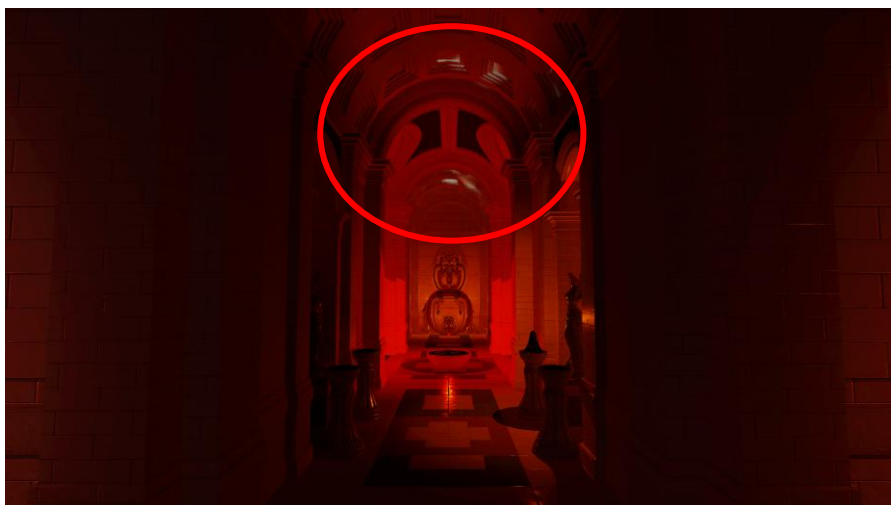
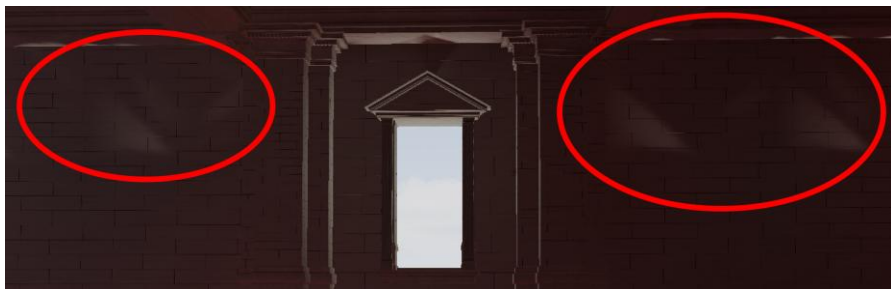
Paper 2

전역 조명 지원 모바일 광선 추적 시스템의 개발 및 성능 프로파일링

Paper 3

Mobile-DDGI: Lightweight Probe-Based Global Illumination via Adaptive Budget Allocation

DDGI 카메라 중심 다중 볼륨 구성 및 적응적 갱신 방식 적용(계속)



빛 샘 문제

한계

- 다중 볼륨 활용에 따라 프로브가 조밀하게 배치되지 않아 특히 실내와 같이 좁고 긴 지형에서 빛 샘 및 그림자 샘 발생
- 프로브 배치 간격을 결정하는 데에는 여전히 상당한 사전 작업이 필요
- PC에서는 우선도 계산 오버헤드로 인해 오히려 성능이 하락함

향후 개선점

- 현재 프레임에 맞추어 계산하기보다 이전 프레임의 결과를 활용함으로써 GPU 계산 부하를 줄여 정렬 등의 연산을 CPU에서 수행
- 볼륨의 크기를 적응적으로 줄이고 늘리는 방안 모색

감사합니다